

1. Relational Algebra (5 Core Operators)

Procedural

User specifies **WHAT** data & **HOW** to retrieve it step-by-step.

- Selection ($\sigma_p(R)$):** Horizontal slice of rows satisfying condition p. Preserves degree, reduces cardinality.
- Projection ($\pi_A(R)$):** Vertical slice of columns. **Automatically removes duplicate rows!** Degree = |A|.
- Union ($R \cup S$):** Merges tuples. Requires **Union Compatibility** (same degree & matching attribute domains).
- Set Difference ($R - S$):** Tuples in R but absent in S. Non-commutative ($R - S \neq S - R$). Union-compatible.
- Cartesian Product ($R \times S$):** Pairs every row. $\text{deg} = m + n$, $\text{card} = m * n$.

2. Derived Joins & Division Operator

Joins & Math

- Theta Join ($R \bowtie_{\theta} S$):** $\sigma_{\theta}(R \times S)$. Equi-join uses equality (=).
- Natural Join ($R \bowtie S$):** Equi-join on common column names + projects out duplicate columns automatically.
- Outer Joins:** Preserves dangling unmatched tuples with NULLs.
 - Left Outer ($R \ltimes S$)
 - Right Outer ($R \rtimes S$)
 - Full Outer ($R \ltimes\!\!\!\bowtie S$)
- Division Operator ($R \div S$):** Solves "FOR ALL / EVERY" queries.

$$R(A, B) \div S(B) = \pi_A(R) - \pi_A((\pi_A(R) \times S) - R)$$

3. Relational Calculus (TRC vs DRC) & Codd's Theorem

Declarative

Tuple Relational Calculus (TRC): $\{ t \mid P(t) \}$. Variable t represents an entire row. Uses quantifiers $\exists$ (Exists) and $\forall$ (For all). De Morgan: $\forall s(P(s)) \equiv \neg(\exists s(\neg P(s)))$.

Domain Relational Calculus (DRC): $\{ \langle x_1, x_2, \dots \rangle \mid P \}$. Variables represent column attribute domains. Basis of visual QBE query languages.

Safe Expressions: A query is safe if it produces a finite set of tuples bounded within the active domain. Unsafe: $\{ t \mid \neg(t \in R) \}$.

CODD'S THEOREM: Relational Algebra = Safe TRC = Safe DRC

4. SQL Data Types & Storage

Storage

- CHAR(n):** Fixed-length string. Padded with trailing spaces.
- VARCHAR(n) / VARCHAR2:** Variable-length string. Space-efficient.
- INT, BIGINT:** Exact whole numbers without fractions.
- DECIMAL(p, s):** Fixed-point precision p with s decimal digits.
- DATE, TIMESTAMP:** YYYY-MM-DD calendar and microsecond timestamps.
- BLOB, CLOB:** Binary & Character Large Objects (Images, PDFs).

5. SQL Commands Classification

Taxonomy

- DDL (Data Definition):** Schema objects create/alter. Auto-committed! **CREATE**, **ALTER**, **DROP**, **TRUNCATE**, **RENAME**.
- DML (Data Manipulation):** Tuples manage. **SELECT**, **INSERT**, **UPDATE**, **DELETE**.
- DCL (Data Control):** Security. **GRANT**, **REVOKE**.
- TCL (Transaction Control):** Integrity. **COMMIT**, **ROLLBACK**, **SAVEPOINT**.

6. DROP vs TRUNCATE vs DELETE Comparison

10-Marker Must

Evaluation Parameter	DROP TABLE	TRUNCATE TABLE	DELETE FROM
Command Category	DDL (Data Definition)	DDL (Data Definition)	DML (Data Manipulation)
Schema Status	Schema + Data destroyed completely.	Data deleted; Schema preserved.	Data deleted; Schema preserved.
WHERE Clause	Not Applicable.	Not supported (all rows purged).	Supported (WHERE id = 5).
Rollback Capability	Cannot rollback (Auto-committed).	Cannot rollback (Auto-committed).	Can rollback (Logged in undo).
Execution Speed	Instant disk deallocation.	Lightning Fast (Resets HWM).	Slower (Row-by-row undo logging).
Triggers Fired	DROP trigger only.	No DELETE triggers fired!	Fires row-level DELETE triggers.

1. SQL DML Operations

DML Commands

- INSERT INTO:** Inserts single/bulk tuples.
`INSERT INTO Emp (ID, Name) VALUES (101, 'Aman');`
- UPDATE:** Modifies existing column values.
`UPDATE Emp SET Sal = Sal * 1.1 WHERE Dept = 'IT';`
Precaution: Missing WHERE updates all rows in table!
- DELETE FROM:** Purges candidate rows.
`DELETE FROM Emp WHERE Resigned = 1;`
- SELECT:** Declarative query data extraction.

2. SQL Integrity Constraints Hierarchy

Constraints

- NOT NULL:** Column cannot store NULL values.
- UNIQUE:** All rows distinct (allows NULLs in standard SQL).
- PRIMARY KEY:** NOT NULL + UNIQUE table identifier.
- FOREIGN KEY:** References parent table primary key.
- CHECK:** Custom condition validation (`CHECK (Salary > 0)`).
- DEFAULT:** Fallback value on empty insert.

3. Foreign Key Referential Actions (ON DELETE / ON UPDATE)

Integrity Rules

1. CASCADE: Deleting/updating parent row automatically deletes/updates corresponding child records.

2. SET NULL: Deleting parent row sets foreign key column in child to `NULL` (Child must allow NULLs).

3. SET DEFAULT: Resets child foreign key to default defined value.

4. RESTRICT / NO ACTION (Default): Rejects parent deletion with an error if dependent child records exist!

4. Physical SQL Clauses Execution Order (Engine Pipeline)

Engine Logic

1. FROM & JOIN → 2. WHERE → 3. GROUP BY → 4. HAVING → 5. SELECT → 6. DISTINCT → 7. ORDER BY → 8. LIMIT

Crucial Rule: WHERE clause executes BEFORE SELECT, so you **CANNOT use column aliases in WHERE!** ORDER BY executes AFTER SELECT, so aliases are perfectly valid in ORDER BY.

5. The 5 Aggregate Functions

Aggregates

- COUNT(*):** Counts total rows including NULL rows.
- COUNT(col):** Counts non-NULL column entries.
- SUM(col):** Sums non-NULL numeric values.
- AVG(col):** Arithmetic mean: Sum / non-NULL count.
- MIN / MAX(col):** Extremum values (numeric/string/date).
- Golden Rule:** All aggregates ignore NULLs (except COUNT(*)).

6. GROUP BY Projection Rule

Grouping

Partitions candidate rows into summary buckets.

Golden Projection Rule: When using GROUP BY, columns in the SELECT list MUST either:

- Be explicitly listed in the GROUP BY clause, OR
- Be wrapped inside an aggregate function (SUM, AVG, etc.).

7. WHERE vs HAVING (Top AKTU 10-Marker Comparison)

Exam Classic

Comparison Parameter	WHERE Clause	HAVING Clause
Operation Scope	Operates on individual rows (tuples) .	Operates on grouped summary buckets .
Execution Timing	Executes BEFORE GROUP BY partitioning.	Executes AFTER GROUP BY aggregation.
Aggregate Functions	Strictly Forbidden (<code>WHERE AVG(sal) > 50k</code> is ERROR).	Designed for aggregates (<code>HAVING AVG(sal) > 50k</code>).
Syntax Example	<code>SELECT Dept, AVG(Sal) FROM Emp WHERE Status='Active' GROUP BY Dept HAVING AVG(Sal) > 60000;</code>	Can appear without GROUP BY (filters whole table aggregate).

1. SQL Joins & Set Operations

Joins & Sets

- INNER JOIN:** Returns only matching records.
- SELF JOIN:** Joins table with itself (Employee & Manager).
- OUTER JOINS:** Retains unmatched dangling rows with NULL.
 - LEFT OUTER • RIGHT OUTER • FULL OUTER.
- UNION vs UNION ALL:**
 - *UNION*: Deduplicates results (internal sort overhead).
 - *UNION ALL*: Preserves duplicates. Extremely fast!
- INTERSECT:** Common rows • **MINUS:** Subtracts rows.

2. Nested vs Correlated Subqueries

Subqueries

- Scalar Subquery:** Returns single value (=, >, <).
- Multi-Row:** *IN*, *> ANY* (> MIN), *> ALL* (> MAX).
- Correlated Subquery:** Inner query references outer row attributes; executes row-by-row (O(N*M) nested loop).
- EXISTS vs NOT EXISTS:** Boolean short-circuit evaluation! Stops immediately upon finding first match. Faster than IN on large indexed tables.

3. SQL Views & Security

Virtual Views

- View:** Virtual table; stores query in data dictionary.
- Benefits:** Column security masking, complex join simplification, logical data independence.
- Updatable View Rules:** Single base table, contains Primary Key, NO aggregates, NO GROUP BY/DISTINCT.
- WITH CHECK OPTION:** Rejects inserts/updates violating the view's WHERE predicate filter.
- Materialized View:** Physically stored snapshot on disk (OLAP).

4. Database Indexes (B-Tree)

Storage Index

- Clustered Index:** Data physically sorted on disk in index order. Leaf nodes contain actual rows. **Only 1 per table** (Auto on PK).
- Non-Clustered:** Separate B-Tree. Leaf nodes contain Key + Row Pointer. Multiple allowed.
- Dense Index:** Index entry for EVERY row (Any table).
- Sparse Index:** 1 entry per disk block (Requires sorted table!).
- Trade-off:** Fast SELECT O(log N) vs slower INSERT/UPDATE.

5. PL/SQL Cursors (Memory Pointer)

PL/SQL

- Implicit Cursors:** Auto-managed by DBMS for all DML. Attributes: *SQL%FOUND*, *SQL%NOTFOUND*, *SQL%ROWCOUNT*, *SQL%ISOPEN*.
- Explicit Cursors (4-Step Lifecycle):**
 1. *DECLARE* (define query) → 2. *OPEN* (execute active set) → 3. *FETCH* (load into variables) → 4. *CLOSE* (free memory).
- Cursor FOR Loop:** Auto opens, fetches & closes cursor.

6. Database Triggers (ECA Model)

Automation

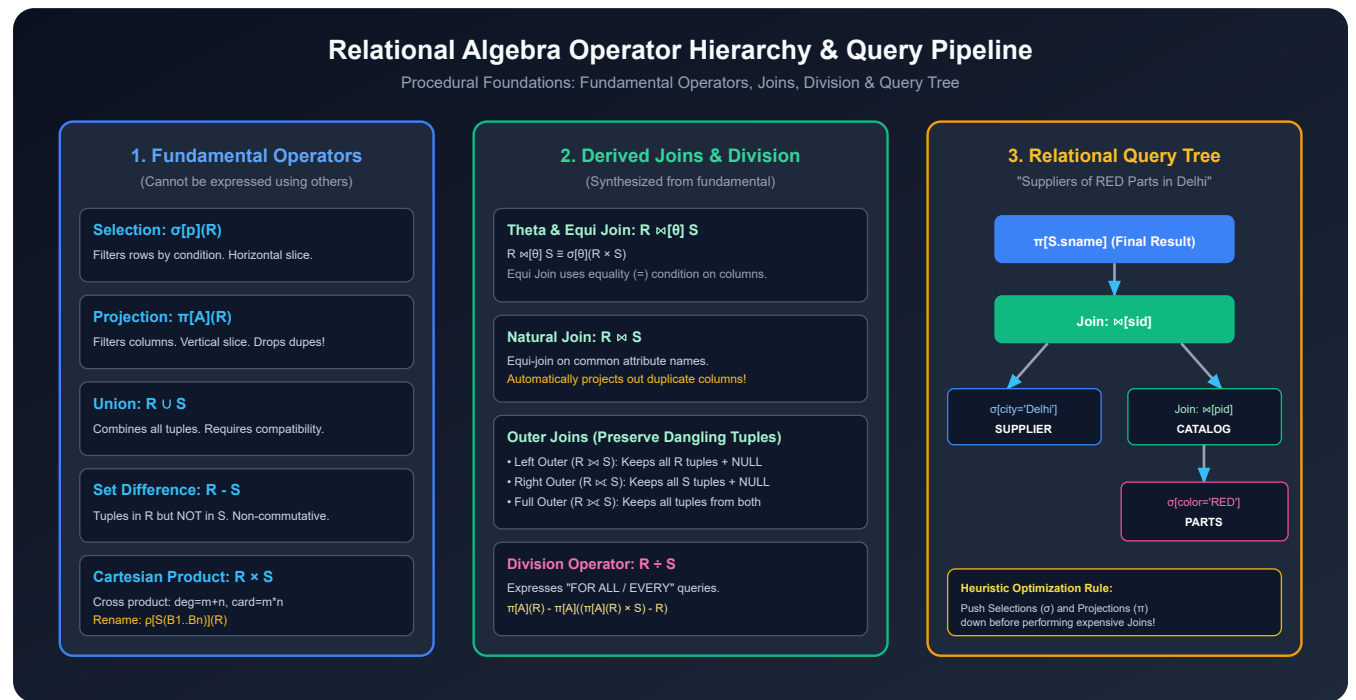
- Event:** INSERT, UPDATE, DELETE on table.
- Timing:** *BEFORE* (validation) vs *AFTER* (auditing/logs).
- Granularity:** *FOR EACH ROW* (Row-level) vs Statement-level.
- Pseudo-records:** *:NEW* (incoming values) and *:OLD* (pre-update values). Reference: *:NEW.Salary*.
- AKTU Example:** Prevent salary reduction:
`IF :NEW.Salary < :OLD.Salary THEN RAISE_APPLICATION_ERROR(-20001, 'Err'); END IF;`

7. Solved AKTU Examination Problem: Suppliers-Parts Database

10-Marker PYQ

Target Query Goal	Relational Algebra Solution	SQL Query Solution
Suppliers of RED parts	$\pi_{\text{name}}(\text{Suppliers} \bowtie \pi_{\text{sid}}(\text{Cat} \bowtie \sigma_{\text{color}=\text{'RED'}}(\text{Parts})))$	<code>SELECT DISTINCT S.sname FROM Suppliers S JOIN Catalog C ON S.sid=C.sid JOIN Parts P ON C.pid=P.pid WHERE P.color='RED';</code>
Suppliers of ALL parts (Division)	$\pi_{\text{name}}(\text{Suppliers} \bowtie (\pi_{\text{sid}, \text{pid}}(\text{Cat}) \div \pi_{\text{pid}}(\text{Parts})))$	<code>SELECT S.sname FROM Suppliers S JOIN Catalog C ON S.sid=C.sid GROUP BY S.sid, S.sname HAVING COUNT(DISTINCT C.pid) = (SELECT COUNT(*) FROM Parts);</code>
Suppliers of NO Red parts	$\pi_{\text{sid}}(\text{Suppliers}) - \pi_{\text{sid}}(\text{Cat} \bowtie \sigma_{\text{color}=\text{'RED'}}(\text{Parts}))$	<code>SELECT sname FROM Suppliers WHERE sid NOT IN (SELECT C.sid FROM Catalog C JOIN Parts P ON C.pid=P.pid WHERE P.color='RED');</code>

Module 01: Relational Algebra Complete Operators and Solved Numericals



Procedural Query Foundation: Relational Algebra specifies WHAT data to retrieve and the exact procedural STEP-BY-STEP pipeline.

1. Fundamental Operators

Fundamental operators are atomic. The 5 core operators are Selection (σ), Projection (π), Union ($\cup$), Set Difference ($-$), and Cartesian Product ($\times$).

Operator	Symbol	Dimension	Key Behavior	Closure Property
Selection	$\sigma_p(R)$	Horizontal (Rows)	Filters tuples based on predicate condition p.	Preserves degree, reduces cardinality.
Projection	$\pi_A(R)$	Vertical (Cols)	Extracts specified columns and automatically eliminates duplicate rows.	Reduces degree to A .
Union	$R \cup S$	Row Concatenation	Combines tuples. Requires Union Compatibility (same degree & domains).	Removes duplicates.
Set Difference	$R - S$	Tuple Subtraction	Tuples present in R but absent in S. Non-commutative ($R - S \neq S - R$).	Union compatible.
Cartesian Product	$R \times S$	All Combinations	Pairs every tuple of R with every tuple of S. $\text{deg} = m + n$, $\text{card} = m * n$.	No condition needed.

2. Derived Joins & Division

Natural Join ($R \bowtie S$) performs an equality join on common columns and drops redundant duplicate columns. Outer joins (Left, Right, Full) preserve non-matching dangling tuples by padding missing values with NULLs.

Division Operator ($R \div S$): Formulated as $\pi_A(R) - \pi_A((\pi_A(R) \times S) - R)$. Used to answer "FOR ALL / EVERY" queries.

3. Solved AKTU Numerical Schema

Suppliers-Parts Database: SUPPLIERS(sid, sname, address), PARTS(pid, pname, color), CATALOG(sid, pid, cost).

```
-- Find names of suppliers who supply some RED part:  
 $\pi[\text{sname}](\text{SUPPLIERS} \bowtie \pi[\text{sid}](\text{CATALOG} \bowtie \sigma[\text{color} = \text{'RED'}](\text{PARTS})))$   
  
-- Find suppliers who supply EVERY part:  
 $\pi[\text{sid}, \text{pid}](\text{CATALOG}) \div \pi[\text{pid}](\text{PARTS})$ 
```

Module 02: Relational Calculus (TRC and DRC)

Relational Calculus: TRC vs DRC Foundations

Declarative Non-Procedural Query Languages, Quantifiers & Codd's Equivalence

Tuple Relational Calculus (TRC)

Filters entire tuples: { t | P(t) }

$$\{ t \mid t \in \text{EMPLOYEE} \wedge t.\text{salary} > 50000 \}$$

"Find all tuples t in EMPLOYEE having salary > 50k"

First-Order Logic Quantifiers:

- Existential Quantifier ($\exists$): $\exists s \in R (P(s))$ - True if at least one exists
- Universal Quantifier ($\forall$): $\forall s \in R (P(s))$ - True if all satisfy P

Equivalence: $\forall s (P(s)) \equiv \neg(\exists s (\neg P(s)))$

Safe TRC Expressions

Query MUST NOT produce infinite tuples!
Unsafe: $\{ t \mid \neg(t \in \text{EMPLOYEE}) \} \rightarrow$ Returns infinite universe!

Domain Relational Calculus (DRC)

Filters individual attribute domain variables: { <x1..xn> | P }

$$\{ \langle e, n \rangle \mid \exists d, s (\langle e, n, d, s \rangle \in \text{EMP} \wedge d = \text{'Sales'}) \}$$

"Get EmpID (e) and Name (n) of employees in Sales dept"

Domain Variables vs Tuples:

- TRC variable represents an entire row/tuple (t.name, t.age).
- DRC variables represent specific column values ((id, name, sal)).

Foundation of QBE (Query-By-Example) visual query builders!

Safe DRC Expressions

All values in result must come from the domain of the database.
Guarantees safety across finite relation states.

Codd's Theorem (Relational Completeness Equivalence)

$\text{Relational Algebra} \equiv \text{Safe TRC} \equiv \text{Safe DRC}$
Every query expressible in Relational Algebra can be written in Safe TRC and Safe DRC, forming the bedrock of modern SQL.

Non-Procedural Power: Relational Calculus specifies WHAT data is needed using mathematical First-Order Logic, without specifying the procedural access path.

1. Tuple Relational Calculus (TRC)

Format: { t | P(t) }. Tuple variable t represents an entire row. Quantifiers $\exists$ (There exists) and $\forall$ (For all) allow joins and universal subset queries.

```
-- Find names of employees in 'Research' earning > 60000:
{ t.name | t ∈ EMPLOYEE ∧ t.dept = 'Research' ∧ t.salary > 60000 }
```

2. Domain Relational Calculus (DRC)

Format: { <x1, x2, ...> | P(x1, x2, ...) }. Variables represent individual column domains.

```
-- Find emp_id and name of employees in 'Sales':
{ <e, n> | ∃d, s (<e, n, d, s> ∈ EMPLOYEE ∧ d = 'Sales') }
```

Safe Queries: Unsafe queries like { t | $\neg(t \in R)$ } return infinite universes. A calculus expression is Safe if all resulting values are bounded within the active domain of the database.

3. Codd's Theorem

Equivalence: Relational Algebra $\equiv$ Safe TRC $\equiv$ Safe DRC. Any database query language matching this power is termed *Relationally Complete*.

Module 03: SQL Foundations, Data Types, and DDL Commands

SQL Command Ecosystem & DDL Structural Lifecycle

Characteristics, Data Types & DDL Commands: CREATE, ALTER, DROP, TRUNCATE

1. SQL Core Characteristics & Types

Essential SQL Data Types & Storage:

- CHAR(n): Fixed-length string. Space padded if shorter. Fast for fixed codes.
- VARCHAR(n) / VARCHAR2(n): Variable-length. Space efficient.
- INT, SMALLINT, BIGINT: Discrete integer values.
- DECIMAL(p, s) / NUMERIC: Fixed-point precision p with s decimals (currency).
- DATE, TIME, TIMESTAMP: Calendar date & fractional second timestamps.
- BLOB, CLOB: Binary & Character Large Objects (Images, PDFs, JSON).

The 4 SQL Command Families:

- DDL (Data Definition): CREATE, ALTER, DROP, TRUNCATE, RENAME.
- DML (Data Manipulation): SELECT, INSERT, UPDATE, DELETE.
- DCL (Data Control): GRANT, REVOKE (Permissions & security).
- TCL (Transaction Control): COMMIT, ROLLBACK, SAVEPOINT.

DDL operations are **AUTO-COMMITTED!** Cannot be rolled back!

2. DDL Structural Commands Deep Dive

CREATE TABLE

```
CREATE TABLE Student (Roll INT PRIMARY KEY, Name VARCHAR(50));
```

Allocates data dictionary metadata and initial disk extent.

ALTER TABLE (Structural Modifications)

- ADD: ALTER TABLE Student ADD Email VARCHAR(100);
- DROP: ALTER TABLE Student DROP COLUMN Email;
- MODIFY: ALTER TABLE Student MODIFY Name VARCHAR(100);
- RENAME: ALTER TABLE Student RENAME COLUMN Roll TO RollNo;

TRUNCATE vs DROP vs DELETE (AKTU Comparison)

- DROP: Deletes table DATA + SCHEMA completely from disk. Free space.
- TRUNCATE: Deletes ALL rows. SCHEMA remains. High speed (resets HWM).
- DELETE: DML command. Deletes specific rows. Generates undo logs.

TRUNCATE is DDL (Auto-commit, no WHERE clause, lightning fast!).

DDL Auto-Commit: DDL commands (CREATE, ALTER, DROP, TRUNCATE) modify the data dictionary directly and are automatically committed!

1. Essential SQL Data Types

Type	Storage	Description
CHAR(n)	Fixed n bytes	Fixed length string padded with trailing spaces. Ideal for fixed-width identifiers.
VARCHAR(n)	Variable length	Dynamic string up to n characters. Space-efficient for names and descriptions.
DECIMAL(p, s)	Exact precision	Fixed-point numeric with p total digits and s fractional digits (e.g. monetary amounts).
TIMESTAMP	Date + Time	Calendar date, hour, minute, second, and fractional microsecond values.

2. TRUNCATE vs DROP vs DELETE

Feature	DROP	TRUNCATE	DELETE
Category	DDL	DDL	DML
Schema State	Completely Removed	Preserved	Preserved
Rollback	No	No	Yes (via ROLLBACK)
WHERE Clause	No	No	Yes
Speed	Instant	Fast (Resets HWM)	Slower (Row logging)

Module 04: SQL DML and Integrity Constraints

SQL DML Operations & Relational Integrity Enforcement

Data Manipulation Commands, Constraints Hierarchy & Referential Cascade Rules

1. DML Operations (Data Manipulation)

INSERT INTO (Tuple Insertion)

```
INSERT INTO Student (Roll, Name) VALUES (101, 'Aman');
```

Bulk Insert: `INSERT INTO Backup SELECT * FROM Student;`

UPDATE (Tuple Modification)

```
UPDATE Employee SET Salary = Salary * 1.1 WHERE Dept='IT';
```

Warning: Missing WHERE updates ALL rows in the table!

DELETE FROM (Row Deletion)

```
DELETE FROM Student WHERE CPI < 5.0;
```

Maintains undo log; triggers foreign key cascade checks.

SELECT (Data Retrieval Expression)

```
SELECT Name, CPI FROM Student WHERE Dept='CSE' ORDER BY CPI DESC;
```

Combines relational algebra σ , π , joins, and aggregation into one declarative query.

2. SQL Integrity Constraints & Cascades

The 6 SQL Constraints Matrix:

- NOT NULL: Column cannot store NULL values.
- UNIQUE: All values in column must be distinct (allows NULLs).
- PRIMARY KEY: NOT NULL + UNIQUE. Table identifier.
- FOREIGN KEY: References parent table primary key.
- CHECK: Custom condition validation (e.g.

CHECK (Salary > 0)).

- DEFAULT: Default fallback value when no value is supplied.

Foreign Key Referential Integrity Actions: ON DELETE / ON UPDATE <ACTION>

1. CASCADE: Deleting/updating parent auto-deletes/updates child rows.
2. SET NULL: Deleting parent sets foreign key in child to NULL.
3. SET DEFAULT: Deleting parent resets child FK to its default value.
4. RESTRICT / NO ACTION: Rejects parent deletion if child row exists! Default action in SQL is RESTRICT (prevents orphaned records).

Data Manipulation & Integrity: DML commands operate on data rows, while integrity constraints guarantee ACID database consistency.

1. SQL Integrity Constraints

Constraint	Rule	Enforcement
NOT NULL	Column cannot be blank.	Table column definition.
UNIQUE	No two rows share the same value.	Unique index created.
PRIMARY KEY	NOT NULL + UNIQUE identifier.	Clustered index created.
FOREIGN KEY	Value must exist in parent Primary Key.	Referential integrity check.
CHECK	Validates custom business rule condition.	Predicate validation.

2. Referential Integrity Actions

When a parent key is deleted or updated: CASCADE propagates changes, SET NULL blanks child foreign keys, and RESTRICT rejects parent deletion to prevent orphaned records.

Module 05: SQL Clauses, Aggregate Functions, and Groupings

SQL Clauses Order of Execution & Aggregation Pipeline

Written Syntax Order vs True Physical Engine Execution Order, GROUP BY & HAVING

Physical SQL Execution Order (Engine)

1. FROM & JOIN
Identifies tables and calculates Cartesian product / Joins.

2. WHERE (Row Filtering)
Filters individual tuples BEFORE grouping. No aggregate functions allowed!

3. GROUP BY (Bucket Creation)
Groups rows having identical values in specified columns into summary buckets.

4. HAVING (Group Filtering)
Filters grouped buckets AFTER aggregation (e.g. HAVING COUNT(*) > 5).

5. SELECT & Expressions
Evaluates selected columns, expressions, aliases and DISTINCT keyword.

6. ORDER BY (Sorting)
Sorts final resulting records ASC / DESC. Column aliases can now be used.

7. LIMIT / OFFSET: Top-N rows

Aggregates & WHERE vs HAVING

The 5 Core Aggregate Functions:

- COUNT(*): Counts total rows including NULLs.
- COUNT(col): Counts non-NULL entries in that column.
- SUM(col): Sums non-NULL numeric values.
- AVG(col): Computes arithmetic mean (ignores NULLs!).
- MIN(col) / MAX(col): Minimum / maximum value.

Golden Rule: All aggregates IGNORE NULLs (except COUNT(*)!)

WHERE vs HAVING (10-Marker Comparison):

- Level: WHERE operates on individual rows; HAVING on grouped buckets.
- Timing: WHERE executes BEFORE GROUP BY; HAVING executes AFTER.
- Aggregates: WHERE cannot use SUM/AVG; HAVING can use all aggregates.
- Can exist without GROUP BY: WHERE yes; HAVING acts as whole table filter.

```
SELECT Dept, AVG(Salary) FROM Employee
WHERE Salary > 30000 GROUP BY Dept
HAVING AVG(Salary) > 60000;
```

Execution Pipeline: The physical database engine executes FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

1. WHERE vs HAVING Comparison

Feature	WHERE	HAVING
Scope	Individual rows before grouping.	Grouped summary buckets after aggregation.
Aggregates	Forbidden (e.g. WHERE AVG(x) is invalid).	Allowed (e.g. HAVING COUNT(*) > 5).
Timing	Filters input tuples early.	Filters groups late in pipeline.

2. Aggregate Functions

All aggregate functions (SUM, AVG, MIN, MAX, COUNT(col)) strictly ignore NULL entries. Only COUNT(*) counts all rows including NULL records.

Module 06: SQL Joins and Set Operations



Data Unification: Joins combine columns from related tables horizontally, while Set Operations append rows from union-compatible queries vertically.

1. SQL Outer Joins

Join Type	Left Table Tuples	Right Table Tuples	Unmatched Value
LEFT OUTER	100% Retained	Only Matches	Padded with NULL
RIGHT OUTER	Only Matches	100% Retained	Padded with NULL
FULL OUTER	100% Retained	100% Retained	Padded with NULL

2. UNION vs UNION ALL

UNION performs an internal sort-merge to eliminate duplicate rows. UNION ALL appends row streams directly without deduplication, offering superior performance.

Module 07: Nested Subqueries and Correlated Queries

SQL Nested Subqueries vs Correlated Subqueries

Independent Evaluation vs Row-by-Row Execution, IN, ANY, ALL, EXISTS

1. Independent Nested Subqueries

Inner query runs ONCE and passes result to outer

Scalar Subquery (Single Value Return)

Uses scalar comparison operators: =, >, <, >=, <=

```
SELECT Name FROM Emp WHERE Salary > (SELECT AVG(Salary) FROM Emp);
```

Inner subquery executes once → returns scalar 50000 → outer query filters.

Multi-Row Subquery Operators:

- IN: Matches any value in returned result list.
- > ANY: Greater than at least ONE value (equivalent to > MIN).
- > ALL: Greater than EVERY value in the list (equivalent to > MAX).

```
SELECT Name FROM Emp WHERE Salary > ALL (SELECT Salary FROM Emp WHERE Dept='Sales');
```

Finds employees earning strictly more than ALL Sales employees.

Performance Characteristics:

Subquery executes exactly once. Result cached in RAM. Complexity: $O(N + M)$.

2. Correlated Subqueries & EXISTS

Inner query references outer table row & executes FOR EACH ROW

Correlated Execution Loop

```
SELECT E1.Name, E1.Salary FROM Employee E1
WHERE E1.Salary > (SELECT AVG(E2.Salary) FROM Employee E2
WHERE E2.DeptID = E1.DeptID); -- Correlation link!
```

"Find employees earning more than their own department average."
If outer table has 10,000 rows, inner subquery runs 10,000 times! $O(N*M)$.

EXISTS vs NOT EXISTS (Boolean Short-Circuit)

- EXISTS: Returns TRUE as soon as FIRST matching record is found!
- Never pulls actual data columns; only verifies existence.

```
SELECT C.CustName FROM Customers C WHERE EXISTS
(SELECT 1 FROM Orders O WHERE O.CustID = C.CustID);
```

Significantly faster than 'IN' when dealing with large subquery sets!

Golden Rule:

Use EXISTS over IN when subquery table is large and indexed!

Nested vs Correlated: Independent subqueries execute once; correlated subqueries evaluate repeatedly for every candidate row of the outer query.

1. Multi-Row Operators

Operator	Logic	Equivalence
IN	Matches any value in the returned list.	= ANY
> ANY	Greater than at least one value in the list.	> MIN(...)
> ALL	Greater than every single value in the list.	> MAX(...)

2. EXISTS Short-Circuiting

EXISTS terminates immediately upon finding the first match, making it superior to IN when querying large correlated tables.

Module 08: SQL Views and Security Abstraction

SQL Views Architecture & Security Abstraction

Virtual Tables, Security Column Masking, Updatability Rules & Materialized Views

1. View Concepts & Architecture

What is a View? (Virtual Table)

- A view stores NO physical data rows on disk!
- Only the SQL query definition is stored in the Data Dictionary.

```
CREATE VIEW IT_Emp AS SELECT EmpID, Name FROM Employee WHERE Dept='IT';
```

3 Fundamental Advantages:

- Security & Data Hiding: Masks confidential columns (Salary, SSN).
- Query Simplification: Hides complex 5-table joins behind a simple view.
- Logical Data Independence: Base table schema changes do not break apps.

```
GRANT SELECT ON IT_Emp TO 'analyst_user';
```

Materialized View (Physical Persistence)

- Physically saves query result on disk (Data Warehouse / OLAP).
- Periodic refresh on schedule: REFRESH COMPLETE ON DEMAND.

Accelerates analytical aggregation queries by 100x!

2. Updatable Views & WITH CHECK OPTION

When is a View Updatable (INSERT/UPDATE)?

A view can be updated IF and ONLY IF:

- It is derived from exactly ONE base table.
- Contains the Primary Key of the underlying table.
- Contains NO aggregate functions (SUM, AVG, COUNT).
- Contains NO GROUP BY, HAVING, or DISTINCT clauses.
- Base table NOT NULL columns without default must be present!

WITH CHECK OPTION (Integrity Guard)

Prevents inserts/updates that violate the view's WHERE predicate!

```
CREATE VIEW Sales_Staff AS SELECT * FROM Emp WHERE Dept = 'Sales' WITH CHECK OPTION;
```

Trying to INSERT an 'HR' employee through Sales_Staff will be REJECTED!

Virtual Abstraction: Views do not store physical rows; they store query definitions providing fine-grained security masking and query simplification.

1. Updatable Views Criteria

A view is updatable only when derived from a single base table without aggregate functions, GROUP BY, DISTINCT, or missing NOT NULL columns.

2. WITH CHECK OPTION

Guarantees that all inserts and updates performed through the view strictly conform to the view's WHERE predicate filter.

Module 09: Database Indexes and Access Paths

Database Indexes Architecture & B-Tree Access Paths

Clustered vs Non-Clustered Indexes, Dense vs Sparse Indexing & Query Tuning

1. Clustered vs Non-Clustered Index

Clustered Index (Primary Data Ordering)

- Data rows are **PHYSICALLY** sorted on disk in index key order!
- Leaf nodes contain the **ACTUAL DATA ROWS** (no pointers needed).
- **Exact ONE Clustered Index allowed per table (data can only be sorted one way!).**
- Auto-created when you declare a **PRIMARY KEY**.

```
CREATE CLUSTERED INDEX idx_roll ON Student(RollNo);
```

Non-Clustered / Secondary Index

- Physical data is **NOT** re-sorted. Index is a separate auxiliary structure.
- Leaf nodes contain Index Key + Row Locator (RID / Pointer to data page).
- **Multiple Non-Clustered Indexes allowed per table (e.g., up to 999).**

```
CREATE INDEX idx_name_city ON Student(Name, City); -- Composite
```

Trade-off: Fast search $O(\log N)$ vs slower INSERT/UPDATE (index maintenance).

2. Dense vs Sparse Indexing

Dense Index

- Index record exists for **EVERY SINGLE** search key value in table.
- Faster search (direct pointer to target record without linear block scan).
- Consumes larger memory and storage space.

Can be built on both sorted and unsorted data files.

Sparse Index (Block Anchors)

- Index record exists only for **SOME** search key values (e.g. 1 per disk block).
- Much smaller index size (easily fits in main memory RAM cache).
- **Strict Requirement: The underlying data file MUST be physically sorted!**

Search: Follow index to block anchor → perform linear scan inside block.

Ideal for primary clustered indexes on massive enterprise datasets.

Indexing Trade-off: Indexes drastically accelerate SELECT query retrieval from $O(N)$ to $O(\log N)$, but add overhead to INSERT/UPDATE/DELETE statements due to index tree maintenance.

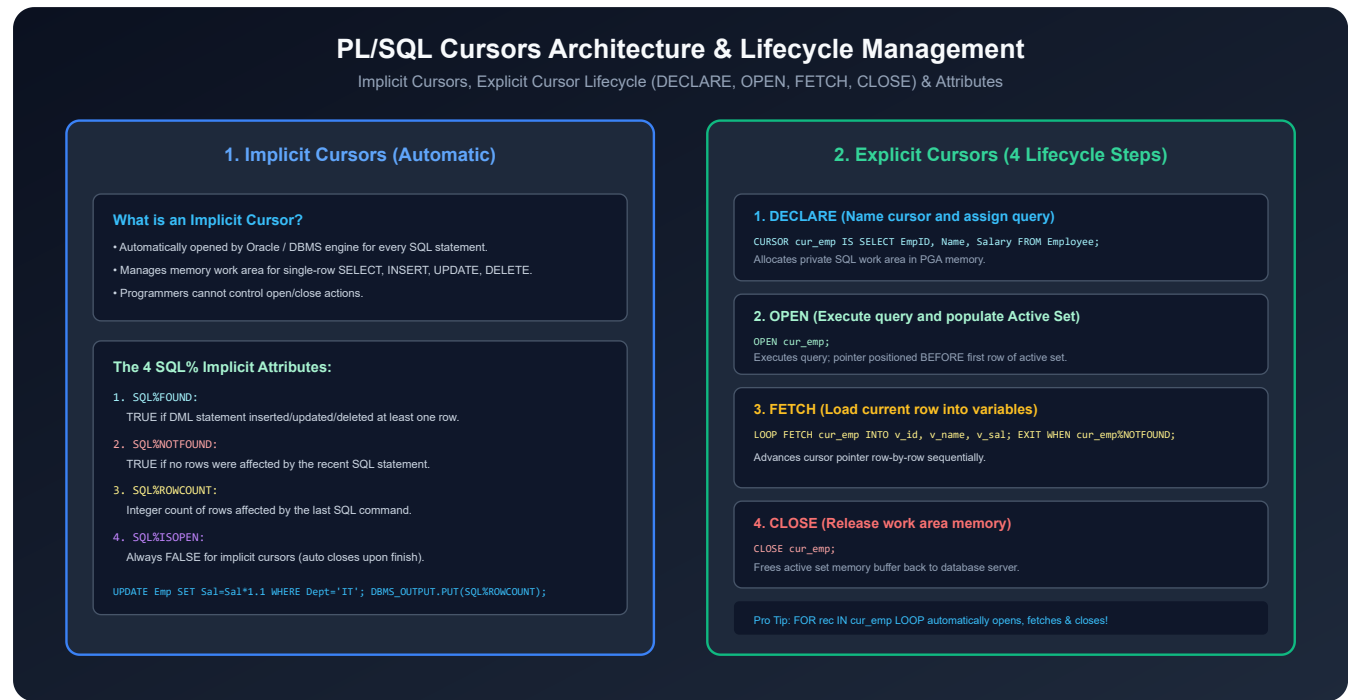
1. Clustered vs Non-Clustered Index

Feature	Clustered Index	Non-Clustered Index
Physical Sort	Physically sorts table data on disk.	Independent B-Tree pointing to data rows.
Count Limit	Exactly 1 per table.	Multiple allowed per table.
Leaf Nodes	Actual data rows.	Search keys + Row Pointers.

2. Sparse Indexing Rule

A Sparse Index stores block anchors (1 entry per block). It is compact enough to fit in RAM, but strictly requires the underlying data file to be sorted.

Module 10: Advanced SQL — PL/SQL Cursors

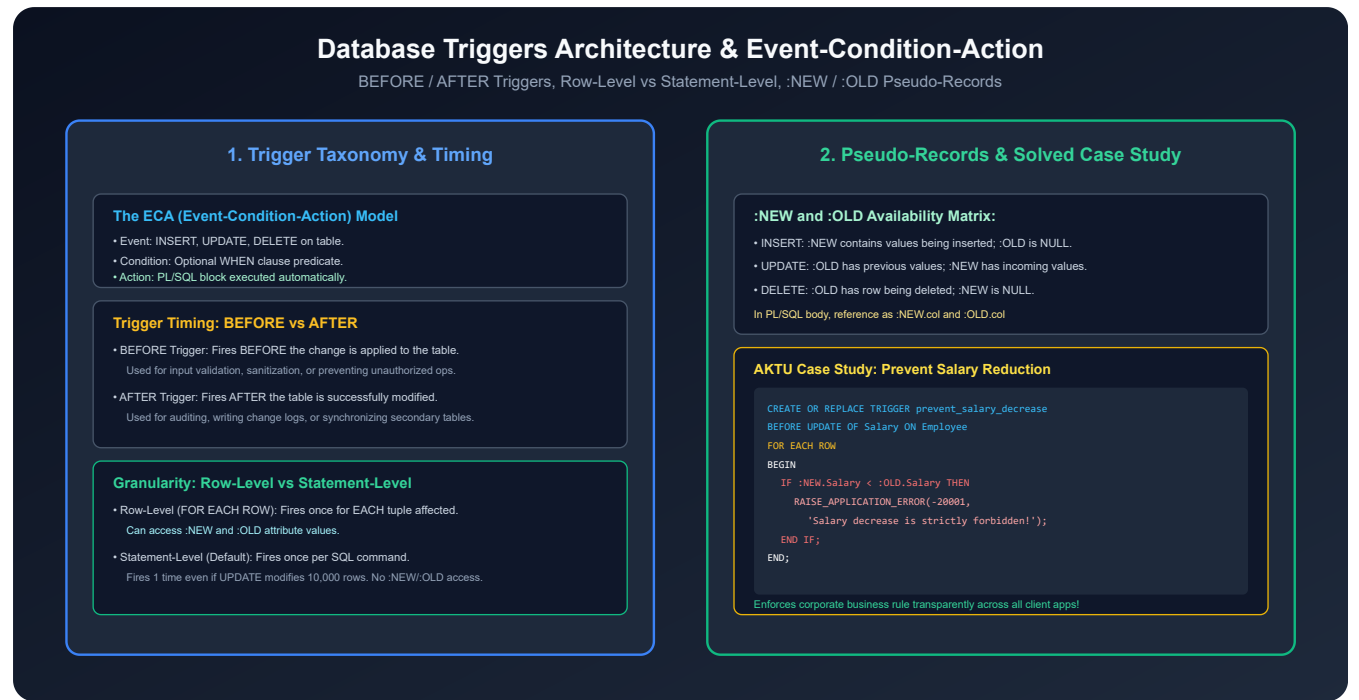


Context Area Management: Cursors manage memory work areas, providing row-by-row navigation across multi-row result sets.

1. Explicit Cursor Lifecycle

Step	Command	Physical Engine Action
1. Declare	CURSOR c IS SELECT...	Allocates cursor definition in PGA.
2. Open	OPEN c;	Executes query, establishes active set.
3. Fetch	FETCH c INTO var;	Advances row pointer and copies columns.
4. Close	CLOSE c;	Deallocates memory work area.

Module 11: Advanced SQL — Database Triggers



Event-Driven Automation: Triggers execute automatically upon DML table events, enforcing business rules and auditing transparently.

1. Trigger Types

Dimension	Options	Application
Timing	BEFORE / AFTER	Validation (BEFORE) vs Auditing (AFTER).
Granularity	Row-Level (FOR EACH ROW) vs Statement-Level	Row-by-row inspection with :NEW/:OLD vs bulk statement audit.

Module 12: Unit 2 AKTU PYQs and Solved Query Bank

AKTU Unit 2 Past 5-Year Solved Examination Papers

Relational Algebra vs SQL Dual Query Bank (Suppliers-Parts, Library, Student-Course)

1. Relational Algebra PYQ Solutions

Suppliers(sid, sname, city) | Parts(pid, pname, color)
Catalog(sid, pid, cost)

Q1: Suppliers of RED parts (AKTU 2021-22)

$\pi[sname](Suppliers \bowtie \pi[sid](Catalog \bowtie \sigma[color='RED'](Parts)))$
Joins Catalog with filtered red parts, then projects supplier names.

Q2: Suppliers who supply ALL parts (AKTU 2022-23)

$\pi[sname](Suppliers \bowtie (\pi[sid, pid](Catalog) \div \pi[pid](Parts)))$
Applies Division Operator ($\div$) to solve universal quantification.

Q3: Suppliers of at least 1 Red AND 1 Green part

$\pi[sname](Suppliers \bowtie (\pi[sid](Cat \bowtie \sigma[color='RED'](P)) \cap \pi[sid](Cat \bowtie \sigma[color='GREEN'](P))))$
Uses Set Intersection ($\cap$) of two supplier ID projections.

Q4: Suppliers who supply NO Red parts (AKTU 2023-24)

$\pi[sname](Suppliers) - \pi[sname](Suppliers \bowtie \pi[sid](Catalog \bowtie \sigma[color='RED'](Parts)))$
Uses Set Difference ($-$) to subtract red part suppliers from all suppliers.

2. SQL Query Dual Solutions

SQL Q1: Suppliers of RED parts (Join Approach)

```
SELECT DISTINCT S.sname FROM Suppliers S
JOIN Catalog C ON S.sid=C.sid JOIN Parts P ON C.pid=P.pid
WHERE P.color = 'RED';
```

SQL Q2: Suppliers of ALL parts (NOT EXISTS / Group By)

```
SELECT S.sname FROM Suppliers S JOIN Catalog C ON S.sid=C.sid
GROUP BY S.sid, S.sname
HAVING COUNT(DISTINCT C.pid) = (SELECT COUNT(*) FROM Parts);
```

Elegant COUNT aggregation approach solves relational division!

SQL Q3: Library Database (Book Issued by Student)

```
SELECT S.Name, B.Title FROM Student S
JOIN Issue I ON S.RollNo=I.RollNo JOIN Book B ON I.ISBN=B.ISBN;
```

SQL Q4: Students who borrowed NO books

```
SELECT Name FROM Student WHERE RollNo NOT IN (SELECT RollNo FROM Issue);
```

Uses NOT IN anti-join subquery to isolate zero-issue students.

Exam Proven Mastery: Relational Algebra numericals and SQL dual query solutions from the last 5 years of AKTU examinations.

1. Suppliers-Parts Relational Algebra Solutions

```
-- Q1: Suppliers of RED parts:
 $\pi[sname](Suppliers \bowtie \pi[sid](Catalog \bowtie \sigma[color='RED'](Parts)))$ 

-- Q2: Suppliers of ALL parts (Division):
 $\pi[sid, pid](Catalog) \div \pi[pid](Parts)$ 

-- Q3: Suppliers of NO Red parts (Set Difference):
 $\pi[sid](Suppliers) - \pi[sid](Catalog \bowtie \sigma[color='RED'](Parts))$ 
```